

# Minard: AI-Human Collaborative Code Understanding

---

**Category:** Plan / Vision **Status:** Draft **Created:** 2026-02-17

"A code review from someone who's read the entire codebase."

## The Core Idea

Minard currently does deep static analysis of codebases — call graphs, import relationships, type signatures, module structure. It visualizes these as interactive maps. But static analysis only reveals *structural* facts. The meaning — why this module exists, what role this function plays, whether this coupling is intentional — requires *semantic* understanding.

An AI can read code and form semantic interpretations. A human has domain knowledge and architectural intent. Neither alone has the full picture. Minard becomes the **sync point** where these two forms of understanding meet, and crucially, where their *disagreements* surface the deepest insights.

## The Workflow

### 1. Load and Analyze

```
minard load ./my-project
```

The existing loader runs: parse source, extract declarations, build call graphs, compute imports, store everything in the database. This is the structural foundation — facts, not opinions.

### 2. AI Reads and Annotates

The AI (Claude, via CLI) reads the codebase and interacts with Minard's query interface:

```
minard query --modules --package my-app
minard query --declarations --module App.Auth --include-source
minard query --call-graph --decl "App.Auth.validateToken" --depth 3
minard render --module App.Auth --format png
```

Based on what it reads, the AI writes annotations back:

```
minard annotate --module App.Auth \
  --kind architecture --value "Authentication boundary"
minard annotate --decl App.Auth.validateToken \
  --kind summary --value "Validates JWT tokens against the shared secret.
  Called on every authenticated route."
minard annotate --decl App.Auth.legacyCheck \
  --kind quality --value "Dead code candidate: only reachable from
  commented-out test"
```

```
minard annotate --module App.Auth --module App.Session \  
  --kind relationship --value "Tight coupling: 14 cross-calls, candidate  
  for merging or extracting shared interface"
```

### 3. Human Reviews

The human opens Minard's visual interface. Instead of a blank map they need to explore, they see an **annotated** codebase:

- Modules badged with architecture labels ("boundary", "adapter", "core domain")
- Declarations flagged with quality signals ("dead code", "high complexity", "untested")
- Relationship indicators ("tightly coupled", "orphaned", "gateway")
- Natural language summaries explaining what each module does

The presentation could take multiple forms (TBD):

- A text overview / executive summary of findings
- A guided tour — a sequence of views, each highlighting a specific insight
- The existing module map, but with AI annotations as badges/filters
- A generated slide deck for team knowledge transfer
- All of the above, depending on context

### 4. Human Confirms, Corrects, Enriches

The human responds to the AI's annotations:

- **Confirms:** "Yes, App.Auth is the authentication boundary" → confidence 1.0
- **Corrects:** "No, legacyCheck isn't dead code — it's called via reflection in production" → AI learns
- **Enriches:** "This module exists because of regulatory requirement X" → context the AI couldn't infer
- **Disagrees:** "These modules shouldn't be merged — the coupling is intentional for deployment reasons"

**The mismatches are the most valuable signal.** When the AI says "adapter" and the human says "core domain", that disagreement reveals:

- The code structure doesn't match the architectural intent
- Either the code should be refactored or the AI's model needs updating
- There's implicit knowledge that isn't captured anywhere in the source

### 5. Shared Understanding

After the review cycle, the codebase has both structural and semantic maps:

- Static analysis facts (calls, imports, types) — always accurate, always current
- AI interpretations — semantic understanding, quality assessments, architecture labels
- Human validations — domain knowledge, intent, corrections
- Disagreement records — the most interesting data of all

This annotated database persists. On the next analysis run, existing annotations are preserved and the AI can build on previous understanding rather than starting fresh.

## The Annotation Schema

```
CREATE TABLE annotations (
  id                INTEGER PRIMARY KEY,
  target_type       VARCHAR NOT NULL,    -- 'declaration', 'module',
'package', 'relationship'
  target_id         INTEGER,             -- FK to
declarations/modules/packages
  target_id_2       INTEGER,             -- For relationships: second target
  kind              VARCHAR NOT NULL,    -- See annotation kinds below
  value             TEXT NOT NULL,
  source            VARCHAR NOT NULL,    -- 'ai', 'human', 'static_analysis'
  confidence        REAL DEFAULT 1.0,    -- 0.0–1.0, AI annotations start
lower
  supersedes        INTEGER,             -- FK to previous annotation this
replaces
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  session_id        VARCHAR              -- Links annotations to a specific
review session
);

CREATE INDEX idx_annotations_target ON annotations(target_type,
target_id);
CREATE INDEX idx_annotations_kind ON annotations(kind);
```

### Annotation Kinds

#### Semantic tags (**semantic\_tag**):

- Architecture role: "boundary", "adapter", "core", "utility", "glue", "shim"
- Pattern: "parser combinator", "smart constructor", "free monad interpreter"
- Purpose: One-sentence summary of what this declaration/module does

#### Quality signals (**quality**):

- "dead\_code", "high\_complexity", "god\_module", "feature\_envy"
- "untested", "test\_only", "deprecated"
- "candidate\_for\_extraction", "candidate\_for\_merging"

#### Architecture (**architecture**):

- Layer assignment: "presentation", "domain", "infrastructure", "cross-cutting"
- Boundary markers: "public\_api", "internal", "adapter"
- Subsystem membership: custom labels grouping modules by function

#### Relationships (**relationship**):

- "tightly\_coupled", "loosely\_coupled", "orphaned"
- "gateway" (single entry point to a subsystem)
- "always\_change\_together" (temporal coupling from git history)

Summaries (summary):

- Natural language description of a module, package, or declaration
- Written by AI after reading source, validated by human

Human context (context):

- Domain knowledge: "Required by GDPR Article 17"
- History: "This was extracted from the monolith in Q3 2024"
- Intent: "Intentionally duplicated for deployment isolation"

What the AI Asks For

Structured Queries (CLI / JSON)

| Query                                         | Purpose                                                           |
|-----------------------------------------------|-------------------------------------------------------------------|
| <code>--modules --package P</code>            | List modules with metadata (LOC, declaration count, import count) |
| <code>--declarations --module M</code>        | All declarations with kind, signature, children, source           |
| <code>--call-graph --decl D --depth N</code>  | Transitive callers/callees                                        |
| <code>--imports --module M</code>             | What M imports and what imports M                                 |
| <code>--unused-exports --package P</code>     | Exported but never imported declarations                          |
| <code>--coupling --module A --module B</code> | Cross-reference count and nature                                  |
| <code>--blast-radius --decl D</code>          | Everything that transitively depends on D                         |
| <code>--complexity --threshold T</code>       | Declarations exceeding complexity metrics                         |
| <code>--annotations --module M</code>         | Existing annotations (previous AI + human)                        |
| <code>--changelog --since S</code>            | What changed and what it touches                                  |

Visual Queries (PNG / SVG)

| Query                                              | Purpose                                  |
|----------------------------------------------------|------------------------------------------|
| <code>--render module-map --package P</code>       | Package treemap with declaration circles |
| <code>--render call-graph --module M</code>        | Force layout of intra-module calls       |
| <code>--render dependency-graph --package P</code> | Module import graph                      |
| <code>--render signature --decl D</code>           | Sigil-rendered type signature            |

The AI interprets images for pattern recognition — seeing clusters, outliers, hairballs — that would take pages of text to describe.

# Visual Interface Implications

## Module View: Badges and Filters

The module signature map gains:

- **Annotation badges** on declarations: small icons/pills for quality signals, architecture roles
- **Filter bar**: toggle filters by annotation kind ("show me dead code", "show me high coupling")
- **Confidence indicators**: AI annotations at low confidence get a subtle "?" marker
- **Mismatch highlights**: where AI and human disagree, show both with visual tension

## Declaration View: Intelligence Card

Replace the source viewer with a **declaration intelligence card**:

- Sigil-rendered type signature (the visual)
- AI summary (one sentence)
- Quality badges
- Usage graph (existing, but enhanced with annotation context)
- "Open in editor" button (jump to VS Code at exact line)
- Annotation history (what the AI said, what the human corrected)

## Package View: Architecture Map

A new view showing the AI's understanding of the package architecture:

- Modules grouped by architecture annotation (layers, subsystems)
- Coupling indicators between groups
- Anomaly badges (modules that seem misplaced)

## Guided Tour Mode

For onboarding (AI or human new to the codebase):

- AI generates a sequence of views, each highlighting a key insight
- "Here's the entry point", "These are the core domain modules", "Watch out for this tightly coupled area"
- Human can annotate the tour: "Actually start here", "Skip this, it's legacy"

# The Fractal Principle

The same annotation + review cycle applies at every level:

| Level       | Static analysis     | AI annotation               | Human validation                     |
|-------------|---------------------|-----------------------------|--------------------------------------|
| Package     | Import graph, LOC   | "This is the HTTP layer"    | "Yes, but it also handles WebSocket" |
| Module      | Declarations, calls | "Authentication boundary"   | "Confirmed"                          |
| Declaration | Signature, callers  | "Smart constructor pattern" | "No, it's a validation function"     |

| Level        | Static analysis      | AI annotation           | Human validation         |
|--------------|----------------------|-------------------------|--------------------------|
| Relationship | Call count, coupling | "Candidate for merging" | "Intentionally separate" |

Open Questions

- **Persistence across loader runs:** When the codebase changes and the loader re-runs, how do annotations survive? Match by name? By content hash? Both?
- **Annotation versioning:** Should the full history be kept, or just current + superseded?
- **Multi-AI sessions:** If different AI models annotate the same codebase, do their annotations compete or complement?
- **Team annotations:** Multiple humans reviewing — how do disagreements between humans surface?
- **Presentation format:** Text summary? Slide deck? Guided tour? All three? Context-dependent?
- **Annotation quality:** How to prevent annotation spam? Rate limiting? Required confidence thresholds?
- **Privacy:** Annotations may contain sensitive architectural knowledge — how to handle in shared/open-source contexts?

Implementation Phases

Phase 1: Query CLI

Add `minard query` and `minard render` commands. Make the existing database queryable from the command line with structured output (JSON, table, PNG). No new analysis — just expose what's already computed.

Phase 2: Annotation Schema

Add the `annotations` table. Add `minard annotate` CLI. Build the read path in the visual interface (show annotations as badges). Human can confirm/reject via the UI.

Phase 3: AI Integration

Build the AI workflow: Claude reads source via `minard query`, writes annotations via `minard annotate`, requests visualizations via `minard render`. Document the protocol so any AI can participate.

Phase 4: Mismatch Intelligence

Track disagreements. Surface them prominently. Build the feedback loop where corrections improve future annotations. This is where the real value emerges.

Phase 5: Guided Tour / Onboarding

AI generates a structured tour of the codebase based on its annotations. Human reviews and refines the tour. The tour becomes a living document that updates as the code and annotations evolve.